

ESLtopia — Test Case Specification

Detailed test cases mapped to the automated Playwright suite · v1.3 · 2026-07-03

Field	Value
Scope	Every automated test currently defined in <code>tests/</code> (13 spec files, 129 defined test cases)
Companion documents	Master Test Plan · Test Scenario Coverage Checklist · <code>docs/qa/test-coverage-todo.md</code>
Legend	UI browser-driven API direct REST/Auth call SKIPPED gated behind an undeployed feature flag

- v1.3 revision:** Added RECU-08 to close the record-sync failure gap flagged in the Test Scenario Coverage Checklist §7 — `syncRecord` logs a failed upsert instead of discarding it silently, but nothing verified that path fired, or that logging doesn't mask the write still being lost. Case count: 128 → 129. Both scenario gaps the v1.1 review surfaced are now closed.
- v1.2 revision:** Added SCHAT-14 and SCHAT-15 to close the teacher-removal cascade gap flagged in the Test Scenario Coverage Checklist §5 (and in this doc's own v1.1 note) — the "remove" happy-path test previously gave the teacher no classes/records to lose, so the cascade delete (`classes.owner_id` / `records.owner_id` both cascade from `auth.users`) was never actually verified. Case count: 126 → 128.
- v1.1 revision:** A full code review pass (app code + all tests + edge functions + migrations + config) fixed 8 issues, none of which changed test-case counts or expected results — **School → Teachers Management** and **Superadmin — Admin Panel & RLS's** `invoke` / `invokeEdgeFn` helpers were deduplicated into `tests/helpers/edgeFunctions.ts` (internal-only change, same assertions), and a stale comment in `superadmin.spec.ts` describing pre-migration DELETE-policy behavior was corrected to match what SADM-12 through SADM-15 already verified. All 126 cases were re-run against the fixed build and pass.
- 16 cases in **School → Teachers Management** and 2 cases in **Superadmin RLS: manage-teacher gate** are currently SKIPPED — the feature is code-complete but not yet deployed (DB migration + edge functions + app). They run automatically once `FEATURE_SCHOOL_TEACHERS=1` is set after deploy.

Module Summary

#	Module	Spec file	Cases
1	Login	<code>tests/auth/login.spec.ts</code>	16
2	Logout	<code>tests/auth/logout.spec.ts</code>	5
3	Registration	<code>tests/auth/registration.spec.ts</code>	12
4	Forgot Password	<code>tests/auth/forgot-password.spec.ts</code>	7
5	Password Reset	<code>tests/auth/password-reset.spec.ts</code>	5
6	Session & Navigation Guards	<code>tests/auth/navigation.spec.ts</code>	3
7	Roles & Permissions	<code>tests/auth/roles.spec.ts</code>	13
8	Superadmin — Admin Panel & RLS	<code>tests/auth/superadmin.spec.ts</code>	19
9	School → Teachers Management	<code>tests/auth/school-teachers.spec.ts</code>	16
10	Classes & Records — Data Layer (REST)	<code>tests/app/classes.spec.ts</code>	15
11	Profile Self-Management	<code>tests/app/profile.spec.ts</code>	5
12	Records Tab — UI	<code>tests/app/records.spec.ts</code>	8
13	PDF Preview Modal	<code>tests/app/pdf-modal.spec.ts</code>	5
Total			129

1. Login

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
LOGIN-01	UI	High	Valid credentials → logged in	Confirmed teacher fixture exists	1. Go to / 2. Fill email + password 3. Submit	"Get started" button visible within 15s
LOGIN-02	UI	High	Wrong password → generic error	Fixture exists	Submit with correct email, wrong password	"Incorrect email or password." shown; sign-in form remains
LOGIN-03	UI	Med	Unregistered email → generic error	None	Submit with a never-used <code>@example.test</code> address	Same generic "Incorrect email or password." (no enumeration)
LOGIN-04	UI	Low	Invalid email format → client validation	None	Submit with <code>notanemail</code>	"Please enter a valid email address." shown; no network call made
LOGIN-05	UI	Low	Empty password → validation error	None	Submit with email only	"Please enter your password." shown
LOGIN-06	UI	Low	Empty email → validation error	None	Submit with password only	"Please enter a valid email address." shown
LOGIN-07	API	High	Password grant → 200 with tokens	Fixture exists	POST <code>/auth/v1/token?grant_type=password</code>	200; <code>access_token</code> , <code>refresh_token</code> present; <code>token_type=bearer</code> ; <code>expires_in>0</code>
LOGIN-08	API	Med	Response includes confirmed user object	Fixture exists	Same POST as LOGIN-07	<code>user.email</code> matches; <code>user.id</code> is a UUID; <code>email_confirmed_at</code> is set
LOGIN-09	API	Med	Access token is a well-formed JWT	Fixture exists	Inspect <code>access_token</code> shape	Matches <code>header.payload.signature</code> pattern
LOGIN-10	API	High	Wrong password → 400	Fixture exists	POST with bad password	HTTP 400
LOGIN-11	API	Med	Unregistered email → 400	None	POST with unused address	HTTP 400
LOGIN-12	API	High	Missing apikey header → 401	Fixture exists	POST without <code>apikey</code>	HTTP 401
LOGIN-13	API	Med	Refresh token grant → new token pair	Fixture exists; prior login	POST <code>grant_type=refresh_token</code> with a valid <code>refresh_token</code>	200; new <code>access_token</code> + <code>refresh_token</code> returned
LOGIN-14	API	Med	Invalid refresh token → 400	None	POST with a bogus string	HTTP 400
LOGIN-15	API	High	Refresh token invalidated after logout	Fixture exists	1. Login 2. Logout with access token 3. Retry refresh grant with the old refresh token	HTTP 400 — refresh token no longer honored
LOGIN-16	UI	High	Unconfirmed user cannot log in	None (signup done in-test)	1. Sign up via public API, no confirmation 2. Attempt login via UI	Generic "Incorrect email or password." — no distinct "unconfirmed" message leaked

2. Logout

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
LOGOUT-01	UI	High	Sign out returns to login screen	Logged-in session	Click "Sign out"	Sign-in form + email placeholder visible within 10s
LOGOUT-02	UI	High	Sign out clears the session	Logged-in session	1. Sign out 2. Reload the page	Still shows sign-in form; no "Sign out" button (no auto-relogin from a stale session)
LOGOUT-03	API	Med	Valid token → 204	Valid access token	POST <code>/auth/v1/logout</code>	HTTP 204 No Content
LOGOUT-04	API	High	Revoked token rejected afterward	Valid access token	1. Logout 2. GET <code>/auth/v1/user</code> with the same token	HTTP 403 (revoked, vs 401 for missing auth)

LOGOUT-05	API	Low	No Authorization header → 401	None	POST logout with no auth header	HTTP 401
-----------	-----	-----	-------------------------------	------	---------------------------------	----------

3. Registration

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
REG-01	UI	High	Signup → confirm → logged in (teacher)	None	1. Sign up, select "Teacher" 2. Confirm via admin API (simulates email link) 3. Log in	"Sign out" button visible — fully authenticated
REG-02	UI	High	Signup → confirm → logged in (school)	None	Same as REG-01, role "School"	"Sign out" button visible
REG-03	UI	Low	Mismatched passwords → validation error	None	Fill form with differing password/confirm	"Passwords do not match."
REG-04	UI	Low	Short first name → validation error	None	First name = "A"	"First name must be at least 2 characters."
REG-05	UI	Low	Short last name → validation error	None	Last name = "B"	"Last name must be at least 2 characters."
REG-06	UI	Med	No role selected → validation error	None	Submit without choosing Teacher/School	"Please select a role."
REG-07	API	High	Valid signup → 200, unconfirmed user	None	POST <code>/auth/v1/signup</code>	200; returned user's email matches; <code>email_confirmed_at</code> falsy
REG-08	API	High	Trigger sets teacher profile role	None	Signup with <code>role: teacher</code>	<code>profiles.role = 'teacher'</code> after the <code>on_auth_user_created</code> trigger fires
REG-09	API	High	Trigger sets school profile role	None	Signup with <code>role: school</code>	<code>profiles.role = 'school'</code>
REG-10	API	High	Superadmin role in metadata is coerced	None	Signup with <code>role: superadmin</code> in metadata	Resulting profile role is <code>teacher</code> , never <code>superadmin</code>
REG-11	API	Med	Weak password → 422	None	Signup with a 3-character password	HTTP 422
REG-12	API	Med	Missing apikey header → 401	None	POST signup without <code>apikey</code>	HTTP 401

4. Forgot Password

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
FPW-01	UI	High	Valid email → confirmation shown	On forgot-password screen	Fill email, click "Send reset link"	"Check your email" + the entered address shown; "Back to sign in" visible
FPW-02	UI	Low	Invalid email format → validation error	On forgot-password screen	Fill <code>notanemail</code>	"Please enter a valid email address."; form stays put, no request sent
FPW-03	UI	Low	"Back to sign in" returns to login	On forgot-password screen	Click "Back to sign in"	Sign-in form with welcome text shown
FPW-04	API	High	Non-existent email → 200	None	POST <code>/auth/v1/recover</code> with an unused address	HTTP 200 (anti-enumeration)
FPW-05	API	High	Registered email → same 200	None	POST recover with any address	HTTP 200 — indistinguishable from FPW-04
FPW-06	API	Med	Missing email field → 4xx	None	POST recover with empty body	HTTP 400–499
FPW-07	API	Med	Missing apikey header → 401	None	POST recover without <code>apikey</code>	HTTP 401

5. Password Reset

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
PWR-01	UI	High	Set new password → old stops working, new works	Fixture user; recovery link generated via admin API	1. Open recovery link 2. Set matching new password twice 3. Try old password 4. Try new password	"Password updated!" then signed out; old password rejected; new password logs in
PWR-02	UI	Low	Mismatched passwords → validation error	On reset screen via recovery link	Fill differing password/confirm	"Passwords do not match."; reset form remains
PWR-03	UI	Low	Weak password → validation error	On reset screen	Fill "weak" in both fields	"Password must be at least 8 characters."
PWR-04	UI	Low	Empty new password → validation error	On reset screen	Submit with both fields empty	"Password must be at least 8 characters."; form remains
PWR-05	UI	High	Recovery link is single-use	Fixture user; recovery link generated	1. Complete reset with the link 2. Revisit the same link	Second visit does not reopen the reset form; sign-in screen shown instead

6. Session & Navigation Guards

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
NAV-01	UI	High	Unauthenticated visit shows login, not the app	No session	Visit / with no cookies/localStorage session	Sign-in form shown; no "Sign out" button present
NAV-02	UI	High	Session persists across reload	Logged-in session	Reload the page	"Get started" landing shown (session restored from localStorage); sign-in form not shown
NAV-03	UI	High	Deactivated account is signed out on reload	Logged-in session; account then set to <code>inactive</code> server-side	1. Log in 2. Deactivate the account via admin API while session is live 3. Reload	App detects <code>status=inactive</code> on the refetched profile, calls <code>signOut()</code> , shows sign-in form

7. Roles & Permissions

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
ROLE-01	UI	High	Signup form offers only Teacher/School	None	Open the signup form	Exactly 2 role cards ("Teacher", "School"); no "Super Admin" text or card anywhere
ROLE-02	API	High	Superadmin signup attempt is coerced	None	POST signup with <code>role: superadmin</code> in metadata	Resulting profile role is <code>teacher</code>
ROLE-03	UI	High	Teacher cannot see/open the Admin portal	Teacher fixture	Log in as teacher, inspect tab bar	No "Admin" tab; "Classes" tab visible
ROLE-04	UI	High	Pending account cannot log in	Fixture set to <code>status=pending</code>	Attempt login via UI	"Your account is pending. Check your email to set your password." shown; sign-in form remains
ROLE-05	UI	High	Inactive account cannot log in	Fixture set to <code>status=inactive</code>	Attempt login via UI	"Your account has been deactivated. Contact your school administrator." shown
ROLE-06	API	Med	Access token contains expected claims	Teacher fixture	Decode <code>access_token</code> payload	<code>email</code> , <code>UUID sub</code> , <code>aud=authenticated</code> , <code>role=authenticated</code> all present
ROLE-07	API	Med	<code>user_metadata.role</code> reflects signup role	Teacher fixture	Decode token payload	<code>user_metadata.role = 'teacher'</code>
ROLE-08	API	Low	Token expiry is in the future	Teacher fixture	Decode token payload	<code>exp</code> > current Unix time
ROLE-09	API	Med	GET <code>/auth/v1/user</code> matches token subject	Teacher fixture	Login, then GET <code>/auth/v1/user</code>	<code>user.id</code> equals token's <code>sub</code> ; email matches
ROLE-10	API	Med	Malformed token → 403	None	GET <code>/auth/v1/user</code> with <code>Bearer not.a.token</code>	HTTP 403

ROLE-11	API	High	Teacher cannot self-promote to superadmin	Teacher fixture	PATCH own profile's role to superadmin via REST with own token	Role remains teacher after the attempt (DB trigger blocks it)
ROLE-12	API	High	School cannot self-promote to superadmin	School fixture	Same as ROLE-11 with a school account	Role remains school
ROLE-13	API	Med	School JWT payload has role: school	School fixture	Decode token payload	<code>user_metadata.role='school'</code> , <code>email</code> , <code>aud=authenticated</code> correct

8. Superadmin — Admin Panel & RLS

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
SADM-01	UI	High	Superadmin sees the Admin tab	Superadmin fixture	Log in, inspect tab bar	"Admin" tab visible
SADM-02	UI	Med	Admin panel lists Delete buttons for other users	Superadmin + a target user fixture	Open Admin tab	At least one visible "Delete" button
SADM-03	UI	Med	No Delete button on superadmin's own row	Superadmin fixture	Open Admin tab, find own ("You") row	Own row has no "Delete" button
SADM-04	API	High	Own profile returns role: superadmin	Superadmin fixture	GET own profile via REST	Single row; <code>role='superadmin'</code>
SADM-05	API	High	Superadmin can list all profiles	Superadmin + peer fixture	GET <code>/rest/v1/profiles</code> with admin token	≥2 rows returned (full-table scan)
SADM-06	API	High	Teacher listing profiles sees only own row	Teacher fixture	GET <code>/rest/v1/profiles</code> with teacher token	Exactly 1 row, matching own email
SADM-07	API	High	Superadmin can read any profile	Superadmin + target fixture	GET a specific profile by id with admin token	1 row; correct email returned
SADM-08	API	High	Teacher cannot read another profile (RLS)	Teacher + target fixture	GET target's profile by id with teacher token	0 rows returned (RLS-filtered)
SADM-09	API	High	Superadmin can change another user's role	Superadmin + target fixture	PATCH target's role to school	HTTP 200; profile role becomes school
SADM-10	API	High	Teacher cannot change another user's role (RLS)	Teacher + target fixture	PATCH target's role with teacher token	Target's role unchanged, regardless of HTTP status
SADM-11	API	High	Superadmin can promote a user to superadmin	Superadmin + target fixture	PATCH target's role to superadmin	HTTP 200; role becomes superadmin
SADM-12	API	Med	Superadmin can delete another profile	Superadmin + target fixture	DELETE target's profile row	Profile no longer exists
SADM-13	API	Med	Superadmin cannot delete their own profile	Superadmin fixture	DELETE own profile row	Own profile still exists after the attempt
SADM-14	API	High	Teacher cannot delete another profile (RLS)	Teacher + victim fixture	DELETE victim's profile with teacher token	Victim's profile still exists
SADM-15	API	High	School cannot delete another profile (RLS)	School + victim fixture	DELETE victim's profile with school token	Victim's profile still exists
SADM-16	UI	High	Role change via Admin panel dropdown	Superadmin + target fixture	Open Admin tab, change target's role dropdown to "school"	Target's DB profile role becomes school within 8s (polled)
SADM-17	API SKIPPED	Med	Superadmin calling manage-teacher → 403	Superadmin + a real teacher created via a school	POST edge function manage-teacher as superadmin	HTTP 403 — function is School-role only
SADM-18	API SKIPPED	Med	Superadmin calling create-teacher → 403	Superadmin fixture	POST edge function create-teacher as superadmin	HTTP 403

SADM-19	—	Placeholder: feature-gated block skipped	—	<code>test.skip</code> when <code>FEATURE_SCHOOL_TEACHERS</code> is unset	Explicitly skipped with a documented reason, not silently omitted
---------	---	--	---	---	---

9. School → Teachers Management Gated behind `FEATURE_SCHOOL_TEACHERS`

All 13 cases below execute only when `FEATURE_SCHOOL_TEACHERS=1` is set (post-deploy of the DB migration + `create-teacher` / `manage-teacher` edge functions). With the flag unset, a single placeholder test runs `test.skip` instead.

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
SCHT-01	API	High	School can create a pending teacher	School fixture	POST edge function <code>create-teacher</code> with email/name	200, <code>ok:true</code> ; new profile has <code>role=teacher</code> , <code>status=pending</code> , correct <code>school_id</code>
SCHT-02	API	High	Teacher account cannot create teachers	Teacher fixture	POST <code>create-teacher</code> as a teacher	HTTP 403
SCHT-03	API	High	School cannot manage another school's teacher	Two school fixtures (A, B) + A's teacher	School B calls <code>manage-teacher</code> (remove) on A's teacher	HTTP 403; teacher profile still exists
SCHT-04	API	High	Deactivate then reactivate toggles status	School + its teacher	1. <code>manage-teacher</code> deactivate 2. reactivate	Status becomes <code>inactive</code> then <code>active</code>
SCHT-05	UI	Med	School sees Employees tab, not global Admin	School fixture	Log in as school, inspect tabs	"Employees" tab visible; "Admin" tab absent
SCHT-06	UI	High	Pending teacher cannot log in	Teacher fixture set to <code>status=pending</code>	Attempt login via UI	Message containing "pending" shown; sign-in form remains
SCHT-07	API	Med	School can remove their own teacher	School + its teacher	POST <code>manage-teacher</code> <code>action=remove</code>	HTTP 200
SCHT-08	API	Low	<code>create-teacher</code> with missing email → 4xx	School fixture	POST <code>create-teacher</code> without <code>email</code>	HTTP 400–499
SCHT-09	API	Low	<code>manage-teacher</code> with unknown action → 400	School + its teacher	POST <code>manage-teacher</code> with <code>action:"fly-to-moon"</code>	HTTP 400–499
SCHT-10	API	Med	Pending teacher self-activates via RPC	Teacher fixture, <code>status=pending</code>	POST RPC <code>activate_my_account</code> with teacher's own token	2xx; profile <code>status</code> becomes <code>active</code>
SCHT-11	API	High	School sees only their own teachers (isolation)	Two schools, each with one teacher	GET <code>/rest/v1/profiles</code> as School A	Contains A's teacher's email; does not contain B's
SCHT-12	API	High	Deactivating a teacher blocks subsequent logins	School + linked teacher	1. Confirm teacher can get a token 2. School deactivates them 3. Retry token grant	Second token request rejected (throws)
SCHT-13	API	Low	<code>create-teacher</code> with duplicate email → error	School fixture + a pre-existing user with the target email	POST <code>create-teacher</code> with an already-registered email	HTTP 400–599
SCHT-14	API	High	Removing a teacher cascades to delete their classes and records	School + linked teacher; a class + record created as that teacher; a superadmin fixture to read rows after the teacher's token is gone	1. Confirm the class + record exist (via superadmin token) 2. POST <code>manage-teacher</code> <code>action=remove</code> 3. Re-check both via superadmin token	Teacher profile gone; the class and its record are both gone (0 rows) — proves the <code>on delete cascade</code> chain, not just the profile deletion

SCHT-15	UI	Med	Remove-teacher confirm dialog discloses class/record deletion	School + linked teacher, logged in as school on the Employees tab	Click "Remove" on the teacher's row; intercept the native <code>confirm()</code> dialog and dismiss it	Dialog message contains "classes", "records", and "cannot be undone"; teacher still exists afterward (dismissed, not confirmed)
---------	----	-----	---	---	--	---

10. Classes & Records — Data Layer (REST)

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
CLS-01	API	High	Teacher can create a class	Teacher fixture	POST <code>/rest/v1/classes</code>	HTTP 201; returned row matches submitted id/name
CLS-02	API	Med	Teacher can list their own classes	Teacher fixture + a class	GET classes filtered by id	1 row, correct name
CLS-03	API	Med	Teacher can rename a class	Teacher fixture + a class	PATCH <code>name</code>	HTTP 200; new name returned
CLS-04	API	Med	Teacher can delete a class	Teacher fixture + a class	DELETE the class, then GET it	DELETE returns 204; subsequent GET returns 0 rows
CLS-05	API	Med	Teacher can add students to a class	Teacher fixture + empty class	PATCH <code>students</code> to a 3-name array	Returned <code>students</code> array matches exactly
CLS-06	API	Med	Teacher can remove a student	Class with 3 students	PATCH <code>students</code> omitting one name	Removed name absent; the other two remain
CLS-07	API	High	Teacher cannot read another teacher's class (RLS)	Two teacher fixtures, one class owned by A	GET the class with Teacher B's token	0 rows returned
CLS-08	API	High	Teacher cannot delete another's class (RLS)	Two teacher fixtures, one class owned by A	DELETE the class with Teacher B's token, then re-check as A	Class still visible to A afterward
CLS-09	API	Med	Teacher can create records for own class	Teacher fixture + a fresh class	POST <code>/rest/v1/records</code>	HTTP 201; <code>class_id</code> matches
CLS-10	API	Med	Teacher can read their own records	Class + record fixture	GET records filtered by <code>class_id</code>	1 row returned
CLS-11	API	Med	Teacher can upsert updated record data	Existing record	POST with <code>resolution=merge-duplicates</code> and new <code>data</code>	HTTP 200/201; updated payment/attendance values reflected
CLS-12	API	High	Teacher cannot read another's records (RLS)	Two teacher fixtures, record owned by A	GET the record with Teacher B's token	0 rows returned
CLS-13	API	High	Deleting a class cascades to its records	Class + record fixture	1. Confirm record exists 2. DELETE the class 3. Re-check the record	Record row is gone after the class is deleted
CLS-14	API	Med	Superadmin can read any teacher's class	Superadmin + teacher-owned class	GET the class with admin token	1 row returned, correct id
CLS-15	API	Med	Superadmin can read any teacher's records	Superadmin + teacher-owned record	GET the record with admin token	1 row returned, correct <code>class_id</code>

11. Profile Self-Management

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
PROF-01	API	Med	Teacher can update own first_name	Teacher fixture	PATCH own profile's <code>first_name</code>	HTTP 200; new value returned
PROF-02	API	Med	Teacher can update own last_name	Teacher fixture	PATCH own profile's <code>last_name</code>	HTTP 200; new value returned
PROF-03	API	High	Teacher cannot change own role	Teacher fixture	PATCH own <code>role</code> to <code>superadmin</code>	Role remains <code>teacher</code> (trigger blocks it)
PROF-04	API	High	Teacher cannot update another's profile	Two teacher fixtures	PATCH peer's <code>first_name</code> with own token	0 rows affected; peer's name unchanged

PROF-05	API	Med	Superadmin can update any user's name	Superadmin + target fixture	PATCH target's <code>first_name</code> with admin token	HTTP 200; new value returned
---------	-----	-----	---------------------------------------	-----------------------------	---	------------------------------

12. Records Tab — UI

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
RECUI-01	UI	Med	Empty state when teacher has no classes	Teacher fixture, no classes	Open Records tab	Empty-state message containing "no classes for records"
RECUI-02	UI	High	Selecting a class lists its students	Class with 2 students	Select the class in the Records dropdown	Table shows exactly 2 rows, both student names visible
RECUI-03	UI	High	Marking present updates row + summary	Class with 2 students	Click the "Absent" toggle for one student	Toggle flips to "Present"; "Present today" summary reads 1 / 2
RECUI-04	UI	Med	"All present" marks every student	Class with 2 students	Click "✓ All present"	Both rows show "Present"; summary reads 2 / 2
RECUI-05	UI	High	Grade set on a student persists	Class with 2 students	1. Select grade "5" for a student 2. Wait for debounced sync 3. Reload and re-select the class	Grade dropdown still reads "5" after reload
RECUI-06	UI	High	Payment toggle updates summary + persists	Class with 2 students	1. Click "Not paid" toggle 2. Verify "Paid" summary = 1/2 3. Wait, reload, re-select class	Toggle still reads "Paid" after reload
RECUI-07	UI	Med	Student profile shows stats and saves a note	Class with 1 student, marked present once	1. Click the student's name 2. Check attendance stat 3. Fill the notes textarea 4. Wait, reload, reopen profile	Modal shows correct name and "1" attendance; note text persists after reload
RECUI-08	UI	High	Failed record save is logged and not silently persisted	Class with 1 student	1. Intercept the records upsert request, fulfill a simulated 500 2. Toggle attendance 3. Wait for the console error 4. Remove the interception, reload	Console logs "Failed to save record"; after reload the attendance mark has reverted — the write never actually landed despite the optimistic UI update

13. PDF Preview Modal

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
PDF-01	UI	High	Generating a worksheet opens the modal with correct info	Teacher fixture, logged in	1. Select topic "am / is / are" 2. Click "Generate worksheet"	Modal visible; title = topic name; subtitle contains "Grade 2" and "exercises"
PDF-02	UI	High	Answer-key toggle shows/hides answers	Modal open with a fill-in worksheet	Click the answer-key switch, then click again	Toggle gains/loses <code>active</code> class; blank answers and the Answer Key block appear then disappear
PDF-03	UI	Med	"New set" regenerates without closing the modal	Modal open	Click "🔄 New set"	Modal remains visible; item count unchanged
PDF-04	UI	High	Close button hides the modal	Modal open	Click the "X" button	Modal removed from the DOM
PDF-05	UI	High	Escape key hides the modal	Modal open	Press Escape	Modal removed from the DOM